Publicación de notas:	24 de junio de 2024
Revisión:	26 de junio de 2024 de 10:30-12:30

Apellidos: _____

Nombre: _____

DNI: _____

FIRMA

PROBLEMA 3 (4 puntos)

Tras mucho tiempo Super Mario se ha cansado de andar por mundos mágicos saltando de plataforma en plataforma y peleando con bichos raros. Además, se ha dado cuenta de que la princesa Peach no necesita ser rescatada (de hecho, ha descubierto que tiene una aventura con Bowser). Fruto de este cansancio y desamor, ha decidido enfocar toda su energía en la empresa de fontanería que comparte con su hermano Luigi. Tras unos años de duro trabajo la empresa ha seguido prosperando y necesita un sistema informático para gestionar las operaciones de la empresa. Para ello, Mario ha contratado a un "informático" que resulta que en verdad era un periodista con un cursillo de Java y sin apenas experiencia. Como era de esperar el diseño es bastante mejorable, pero seguro que un estudiante de informática con amplios conocimientos sobre principios de diseño de software como tú podrá mejorarlo.

Apartado 3.1 (2 puntos)

Puedes ver un fragmento del diagrama de clases de diseño realizado en la Figura 1.

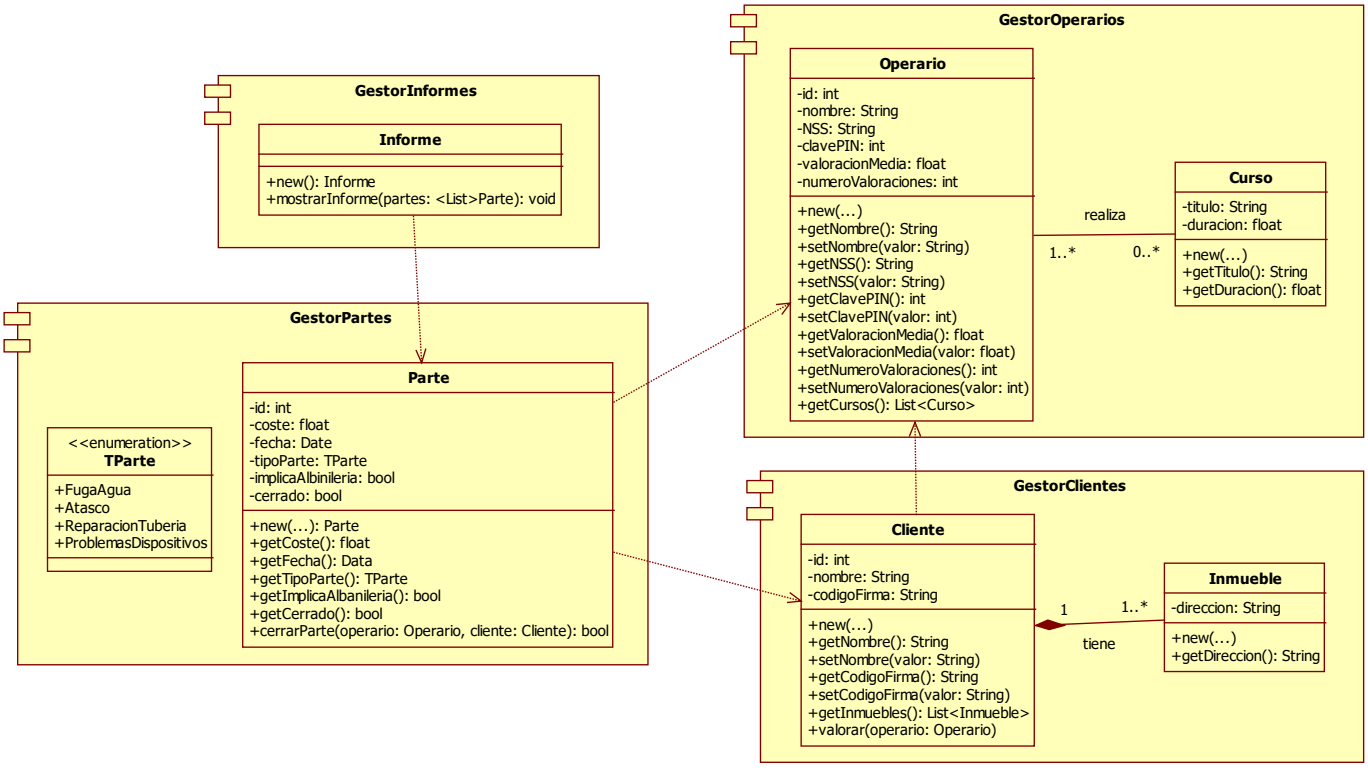


Figura 1. Fragmento del diagrama de clases de diseño

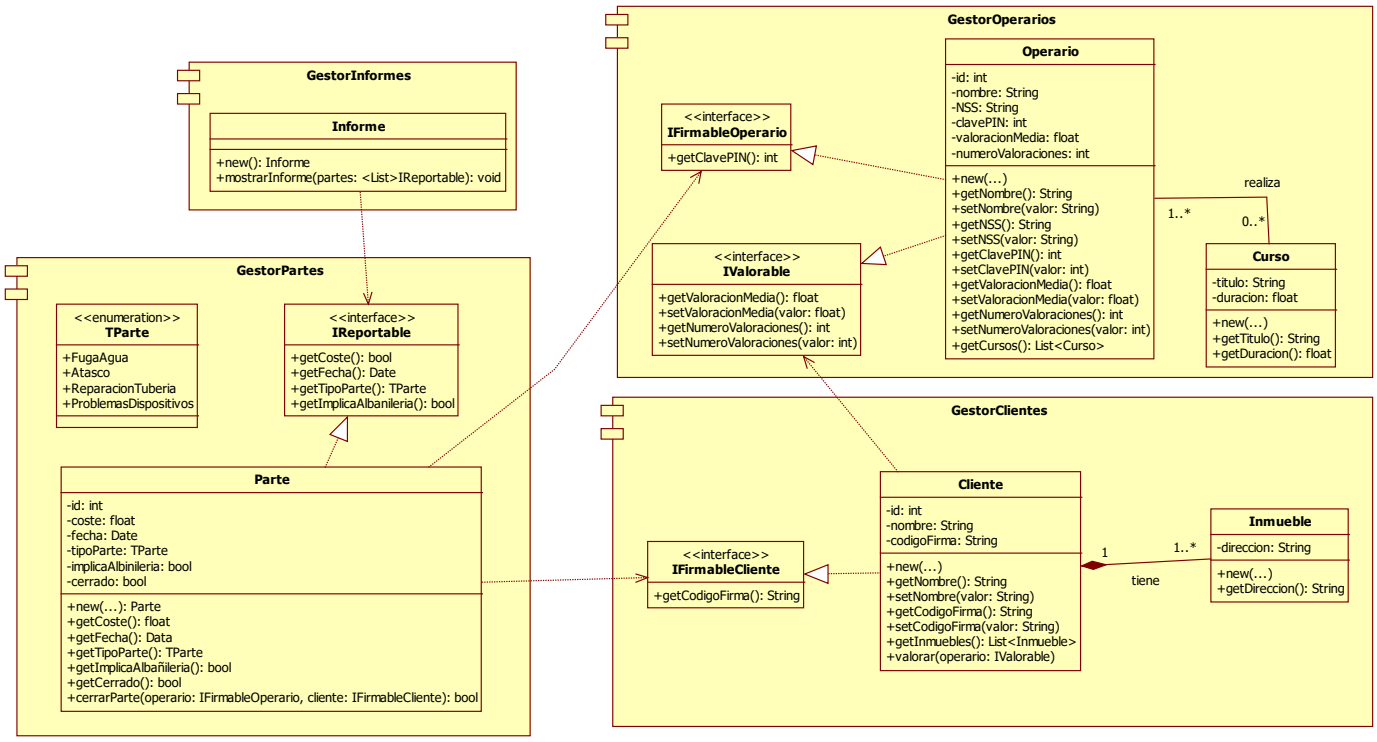
Para comprender las relaciones entre los componentes de este diagrama debes considerar lo siguiente:

- Un informe debe mostrar cierta información de los partes, concretamente en los informes se debe reportar para cada parte: el coste, la fecha, el tipo, y si fueron necesarios trabajos de albañilería.
- Para cerrar un parte es necesario un proceso validación en el que se debe conocer la clavePIN del operario y el código de firma del cliente.
- Un cliente podrá valorar al operario que le ha atendido. En este proceso de valoración se debe conocer la valoración media y el número de valoraciones del operario. Estos datos serán actualizados durante el proceso de valoración teniendo en cuenta la valoración introducida por el cliente (que será un número entero de 1 a 5).

Modifica el diagrama de clases de diseño proporcionado en el enunciado a fin de **reducir el acoplamiento** entre componentes lo máximo posible y de la forma más adecuada posible. Indica también los **principios de diseño** que has aplicado en el siguiente recuadro.

Solución (apartado 3.1)

Principios de diseño aplicados: Inversión de dependencias, segregación de interfaces.


GestorInformes
Informe
+new(): Informe
+mostrarInforme(partes: <List>IReportable): void
GestorOperarios
<<interface>>
IFirmableOperario
+getClavePIN(): int
Operario
-id: int
-nombre: String
-NSS: String
-clavePIN: int
-valoracionMedia: float
-numeroValoraciones: int
+new(...)
+getNombre(): String
+setNombre(valor: String)
+getNSS(): String
+setNSS(valor: String)
+getClavePIN(): int
+setClavePIN(valor: int)
+getValoracionMedia(): float
+setValoracionMedia(valor: float)
+getNumeroValoraciones(): int
+setNumeroValoraciones(valor: int)
+getCursos(): List<Curso>
realiza
1..*
0..*
Curso
-titulo: String
-duracion: float
+new(...)
+getTitulo(): String
+getDuracion(): float
<<interface>>
IValorable
+getValoracionMedia(): float
+setValoracionMedia(valor: float)
+getNumeroValoraciones(): int
+setNumeroValoraciones(valor: int)
GestorPartes
<<enumeration>>
TParte
+FugaAgua
+Atasco
+ReparacionTuberia
+ProblemasDispositivos
<<interface>>
IReportable
+getCoste(): bool
+getFecha(): Date
+getTipoParte(): TParte
+getImplicaAlbanileria(): bool
Parte
-id: int
-coste: float
-fecha: Date
-tipoParte: TParte
-implicaAlbinileria: bool
-cerrado: bool
+new(...): Parte
+getCoste(): float
+getFecha(): Data
+getTipoParte(): TParte
+getImplicaAlbañileria(): bool
+getCerrado(): bool
+cerrarParte(operario: IFirmableOperario, cliente: IFirmableCliente): bool
GestorClientes
<<interface>>
IFirmableCliente
+getCodigoFirma(): String
Cliente
-id: int
-nombre: String
-codigoFirma: String
+new(...)
+getNombre(): String
+setNombre(valor: String)
+getCodigoFirma(): String
+setCodigoFirma(valor: String)
+getInmuebles(): List<Inmueble>
+valorar(operario: IValorable)
1
1..*
tiene
Inmueble
-direccion: String
+new(...)
+getDireccion(): String

Apartado 3.2 (2 puntos)

La empresa ha seguido evolucionando y ahora también ofrece servicios de electricidad. Mientras que para los trabajos de fontanería es necesario conocer el tipo de labor realizada (específica del campo de la fontanería) y si esta implicaba tareas de albañilería, para los trabajos de electricidad es necesario conocer el tipo de labor realizada (específica del campo de la electricidad) y el código CUPS del inmueble en el que se ha realizado el trabajo. Esto debe ser considerado en los informes, que ahora podrán contener información relativa a trabajos tanto de fontanería como de electricidad. En la figura 2 puedes ver el fragmento del diagrama de clases de diseño que propone el "informático" y en el siguiente recuadro puedes ver el código Java correspondiente al método "`mostrarInforme`" de la clase Informe.

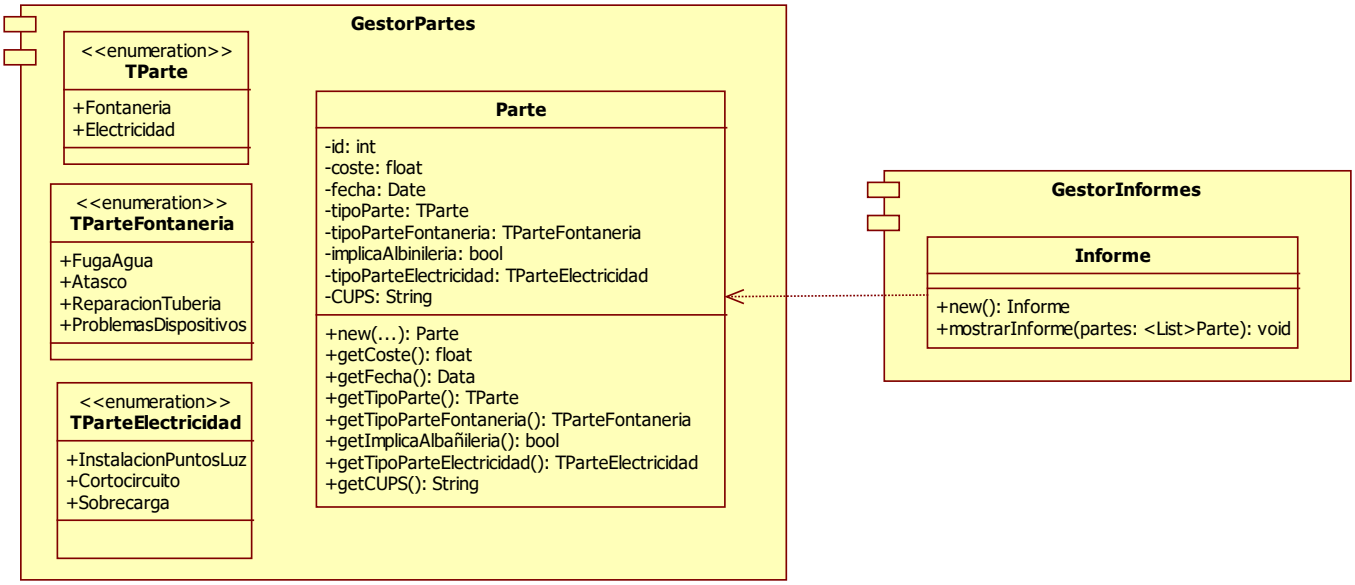
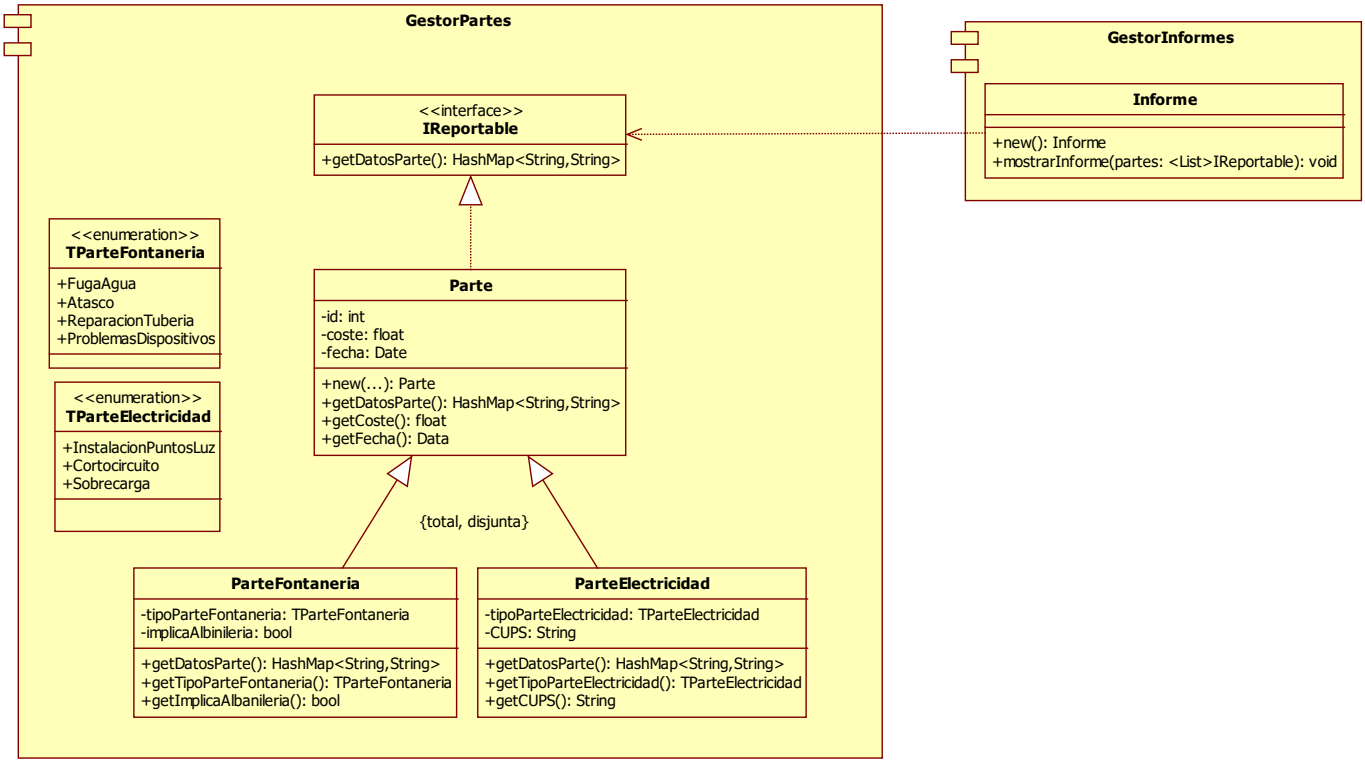


Figura 2. Fragmento del diagrama de clases de diseño

```
public void mostrarInforme(List<Parte> partes) {
    for (Parte p : partes){
        if (p.getTipoParte() == "Fontaneria"){
            System.out.println("Parte de fontaneria: " + p.getTipoParteFontaneria());
            System.out.println("Fecha/Coste: " + p.getFecha() + "/" + p.getCoste());
            if (p.getImplicaAlbanileria())
                System.out.println("Implica albañileria");
        } else { // TipoParte == "Electricidad"
            System.out.println("Parte de electricidad: " + p.getTipoParteElectricidad());
            System.out.println("Fecha/Coste: " + p.getFecha() + "/" + p.getCoste());
            System.out.println("Código CUPS del inmueble: " + p.getCUPS());
        }
    }
}
```

Uno de los problemas de este diseño es que no es extensible y está altamente acoplado. Es decir, si en el futuro Mario y Luigi decidiesen ofrecer nuevos servicios que diesen lugar a nuevos tipos de parte (de pintura, por ejemplo) que tuviesen ciertas particularidades (por ejemplo, cantidad de pintura empleada), sería necesario modificar también la clase Informe. **Lo que se necesita es que el componente GestorInformes pueda mostrar informes de los distintos tipos de parte sin conocer los diferentes tipos de partes existentes ni depender de los detalles de implementación del componente GestorPartes**.

Elabora, partiendo del diagrama de clases de diseño proporcionado en el enunciado, un diagrama de clases de diseño que satisfaga los nuevos requisitos a fin de **incrementar la extensibilidad** del sistema y **reducir el acoplamiento** entre componentes. Indica también los principios de diseño que has aplicado en el siguiente recuadro.

Solución (apartado 3.2)

Principios de diseño aplicados: Abstracción por especialización, responsabilidad única, cohesión, abierto/cerrado, sustitución de Liskov, e inversión de dependencias.